webTemplate — Project Guide

Universal CMS backend for service-based client websites

A scaled-down, WordPress-like CMS built on Laravel 13 + Vue 3 + Inertia + Tailwind v4. The project has two main parts:

1. **Public Website** — the client-facing site, rendered from JSON pages + widgets. Custom frontends are built per client; the backend stays generic.
2. **Admin Panel** — where the client (or our team) manages pages, blog, media, collections, menus, settings, and the site's modules.

Treat it like WordPress for our own service clients: the editor's daily workflow is *write content → drop widgets on a page → publish → check enquiries*, and the developer's workflow is *toggle modules → build the client's widget components → theme → launch*.

> Known bugs and pending polish are tracked separately in `feedback.md` (F1–F13). This guide describes what the template **ships today**.

Part 1: Public Website

1. Dynamic Pages (the core)

- Every page is a JSON file: `data/pages/{slug}.json` (git-diffable, deploys with code — no DB rows for page content)
- Page shape: `title` / `status` (published|draft) / `seo` block / `widgets[]`
- Routing: `/` maps to the `home` slug; `/{slug}` catches all published pages; drafts and missing files 404
- A page is a stack of **widgets** rendered top-to-bottom by `Public/DynamicPage.vue` → one Vue component per widget type in `resources/js/Components/Widgets/`
- Unknown widget types are skipped silently — per-client frontends can override/extend widget components without breaking the backend contract

2. Widget types (13 shipped)

Static widgets (content lives in the page JSON):

- **Hero** — page H1, subtitle, background/side image, CTA buttons (renders the page's single H1; all other widgets start at H2)
- **Rich Text** — TipTap-authored HTML block
- **Feature Grid** — repeater of icon + title + text cards
- **Stats** — repeater of number + label counters

- **Call to Action** — banner with heading + button
- **Image** — single image with caption
- **Gallery** — repeater of images
- **Contact Form** — the public enquiry form (name, email, phone, subject, message)
- **Custom HTML** — raw HTML escape hatch (maps/embeds; lazy-load guidance applies)

Collection widgets (data pulled live from modules via `WidgetDataResolver`):

- **Testimonials** — latest or hand-picked testimonials
- **FAQs** — page-wise: defaults to the *current page's* FAQs, falls back to global; also emits FAQPage schema
- **Team** — team member cards
- **Latest Posts** — recent blog posts

3. Blog

- `/blog` — paginated index with featured images
- `/blog/category/{slug}` — category archives (includes descendant categories), category name/description header
- `/blog/{slug}` — post page: TipTap HTML body, featured image, categories, tags, author, dates
- BlogPosting + BreadcrumbList JSON-LD emitted automatically
- Category chips on cards link to archives

4. Collection pages (module-gated)

- `/careers` + `/careers/{slug}` — job listings with detail pages (JobPosting schema)
- `/case-studies` + `/case-studies/{slug}` — portfolio/case-study listings with detail pages
- Testimonials, FAQs, Events, and Team have **no dedicated public routes** — they appear on pages through collection widgets (deliberate: page composition stays in the editor's hands)

5. Forms & lead capture

- **Contact form** (widget) — honeypot spam guard, international phone validation (`propaganistas/laravel-phone` with a default-country setting), stores the lead as an **Enquiry** *before* queuing the notification email (mail failure never loses a lead)
- **Newsletter signup** — stores subscribers, deduplicated

6. Navigation, header & footer

- Menus are DB-driven with **nesting** (parent/child) and header/footer locations; shared to the frontend as a ready-made tree (active + sorted only)
- Header & footer layout (logo, CTA button, footer columns, copyright with `{year}` / `{site_name}` tokens, social toggle) lives in `data/header.json` / `data/footer.json` → shared `layout` prop

7. SEO (RankMath-parity pack)

- Self-referencing **canonical** tags on every page; per-page/per-post canonical override
- **Title template** (`%title% – %site_name%`) applied when no explicit meta title
- Full social meta: og:title/description/image overrides with sensible fallbacks, `twitter:card=summary_large_image`, article published/modified times on posts
- **Automatic schema**: Organization (site-wide), LocalBusiness (settings-driven builder: type, address, geo, opening hours), BlogPosting, BreadcrumbList, FAQPage (via FAQ widget), JobPosting (careers)
- **XML sitemap** at `/sitemap.xml` — real lastmod, `<image:image>` for featured images, excludes noindex/draft content, cached 24h, regenerate button in the cache panel
- Dynamic `robots.txt`; site-wide noindex kill-switch (`SEO_INDEXABLE` env + settings banner)
- Per-page/per-post: meta title, description, noindex, raw JSON-LD field
- **Redirect engine**: manual 301/302 manager, automatic 301s on slug rename, 404 logging

8. Analytics & consent

- GA4 + GTM injection from settings (consent-aware loading)
- Cookie-consent banner with configurable text
- Custom Code module for arbitrary head/body scripts per page scope

9. Theme (admin ↔ public synced)

- Settings → Theme: primary color (full 7-step brand palette generated from one hex), font (curated bunny-fonts list), border radius
- Emitted as `:root` CSS variables in the Blade shell — admin panel and public site restyle together, response cache busts automatically on save

10. Performance (WP-Rocket-style pack)

- **`AppImage` component** — single image discipline point: srcset/sizes from media variants (thumb 400 / md 1200 / original), width/height attributes (no CLS), lazy + async decoding by default, `eager` + `fetchpriority=high` for above-the-fold
- **LCP preload** — first visible widget's hero image gets `<link rel="preload">`
- Gzip/Brotli compression + 1-year expires headers in `.htaccess`
- WebP conversion + EXIF stripping at upload; `md` / `thumb` variants
- Full-page **response cache** (7-day TTL, database store — inode-frugal on shared hosting), auto-busted by every content save
- Vite-hashed assets, Tailwind purge, bunny-fonts preconnect

11. Other public plumbing

- Auth pages: login, forgot/reset password (admin access; no public registration)
- Custom error pages (500 / 503 / database-down) that work even when Vue can't boot

- RSS feed for the blog
- Fully responsive; toasts/flash messages; 404 page

Part 2: Admin Panel

The sidebar is **generated from module manifests** (never hardcoded), permission-filtered per user, grouped WordPress-style: **Content / Collections / Inbox / Appearance / System**, with live count badges (unread enquiries, unseen subscribers).

1. Dashboard

- Content count tiles + quick actions
- **Global search** (⌘K) across posts, pages, careers, case studies, users — modules register their own searchable config
- Notification bell (read / read-all)
- "Clear page cache" quick action

2. Pages (Content)

- WP-style pages table: title, slug, status, last updated
- **Widget editor**: add widgets from a grouped palette (static/dynamic), collapse/expand cards, show/hide toggle per widget, ↑/↓ reorder, remove
- Field renderer maps registry field types (text, textarea, richtext, image, link, boolean, number, select, repeater, collection) to the shared form atoms — every field has a placeholder
- Right column: page settings (title, slug, status) + SEO section (meta title/description with live length counters, canonical, OG overrides, og:image via media picker, noindex, JSON-LD)
- **Floating save button** (dirty-state aware, always visible while scrolling)
- Create / rename (auto-301 redirect) / delete pages; slug validated against existing files and claimed routes

3. Blog (Content)

- **Posts**: TipTap **block editor** — slash-command block menu (headings, lists, quote, image, divider, HTML), bubble toolbar for inline marks, media-library image insertion; stores HTML
- Featured image via media picker; excerpt; draft/published; publish permission separate from edit
- **Categories**: WordPress-style — many-to-many, hierarchical (parent/child), default "Uncategorized" fallback, single-screen admin (add form + indented tree with post counts), inline "+ add category" from the post editor
- **Tags**: inline CRUD
- Per-post SEO section (same fields as pages) + content checklist (focus keyword, title/description length, alt text, H2/internal-link checks)

4. Media Library (Content)

- Grid browser with search, MIME-type filter, pagination
- Upload (multi-file), **bulk select + bulk delete**, edit alt text
- Pipeline: MIME whitelist (JPEG/PNG/WebP/GIF/PDF — **SVG deliberately blocked**), WebP conversion, EXIF strip, `md` / `thumb` variants, width/height recorded
- `AppMediaPicker` used by every image field in the panel: upload inline *or* choose from the library overlay
- Import-from-URL endpoint; `media:prune` command reports/deletes orphaned files and dead rows
- Site logo + favicon settings actually render (header + `<link rel="icon">`)

5. Collections

Each is a toggleable module with the same CRUD pattern (DataTable index → create/edit forms → activity-logged, cache-busting saves):

- **Testimonials** — author, role/company, quote, rating, photo
- **FAQs** — question/answer, **page assignment** (each FAQ belongs to a page or is global; the FAQ widget scopes to the current page)
- **Events** — title, dates, location, description
- **Team** — name, role, photo, bio, socials
- **Case Studies** — title, client, featured image, body, published state
- **Careers** — job title, location, type, description, active state

6. Inbox

- **Enquiries** — every contact-form submission stored: unread rows highlighted, read/unread toggle, bulk mark-read, search + read/unread filter, detail view (marks read), reply via mailto, delete; **unread-count badge** in the sidebar
- **Newsletter** — subscriber list, search, CSV export, delete; unseen-count badge

7. Appearance

- **Menus** — location tabs (Header/Footer), nesting via indent/outdent + ↑/↓ reorder, inline edit, add Pages/Posts from content or custom links, active toggle
- **Header & Footer** — logo (media picker), header CTA, footer columns (repeater), copyright, social visibility

8. System

- **Settings** — tabbed, driven by DB groups (a new seeded group = a new tab automatically):
  - *General*: site name, description, logo, favicon
  - *Contact*: email, phone, default phone country, address
  - *Social*: WhatsApp, Facebook, Twitter/X, Instagram, LinkedIn, YouTube
  - *SEO & Analytics*: default OG image, noindex banner (read-only `SEO_INDEXABLE` state), title template, LocalBusiness schema builder, GA4, GTM, cookie-consent text

    - *Theme*: primary color, font, radius
    - Secret-ish values encrypted at rest; only a whitelist of keys ever reaches the browser
- **Users & Roles** — user CRUD, role assignment (spatie/laravel-permission); permissions are auto-synced from module manifests (`PermissionSyncer`)
- **Modules** — the WordPress-plugins screen: enable/disable/reinstall any module, hide from nav, health status + clear-health, delete (with fault isolation — a broken module can't take the panel down)
- **Redirects** — 301/302 manager + 404 log (turn a logged 404 into a redirect in one click)
- **Custom Code** — head/body script snippets with enable toggle
- **Audit Log** — who changed what: create/update/delete on all content models (dirty-attributes only), plus login/logout events; filterable by user/model/event
- **Cache panel** — per-layer clearing (Pages / Sitemap / Settings / Modules / Redirects / Views / Everything — never a blind `Cache::flush()`), last-cleared timestamps, **regenerate sitemap** button

9. Admin cross-cutting

- Role-based access on every route *and* every nav item (server-side filtered)
- Flash/toast contract for all saves; validation errors inline per field
- Profile screen (name, email, password)
- Shared component kit: `AppFormField`, `AppInput`, `AppTextarea`, `AppSwitch`, `AppMediaPicker`, `AppBlockEditor`, `AppFloatingSave`, `AppFileInput`, `FormShell`, DataTable — new CRUD screens compose these, never bespoke inputs

---

Part 3: Platform & Developer Features

1. Module system (the differentiator)

- Every feature is a module that can be toggled from the dashboard — **physical** modules in `app/Modules/{Name}/` (own migrations, models, controllers, Vue pages, manifest) or **virtual** modules declared in `config/modules.php`
- A manifest declares: name, nav group, nav entries (+ optional badge resolver), permissions, feature-flag fallback, dependencies, searchable config
- Enabling/disabling flows through everywhere automatically: sidebar, permissions, public routes, sitemap, widget sources
- `rescue()` fault isolation: one broken module logs a health error instead of crashing the app

2. Content storage model

- **Pages = JSON files** (`data/pages/`, `data/header.json`, `data/footer.json`) — versioned in git, deploys atomically with code
- **Collections/blog/settings = MySQL** — client-editable data lives in the DB

- This split is deliberate: page structure is a developer/deploy concern; content entries are a client concern

3. Type safety across the stack

- PHP DTOs (`spatie/laravel-data`, `#[TypeScript]`) auto-generate `types.d.ts` — backend shapes are compile-checked in Vue
- Widget registry generates `widgets.d.ts` (`php artisan widgets:types`) — widget `data` props typed end-to-end
- `pnpm build` runs `vue-tsc` — the type gate for every frontend change

4. CLI toolbox

Command	Purpose
`php artisan template:init`	First-run wizard: site name, admin user, migrate + seed
`php artisan template:doctor`	Pre-launch health check (env, cache config, indexability, dump binary, inodes)
`composer dev`	Serve + queue + logs + Vite in one command
`php artisan typescript:transform`	Regenerate TS types from DTOs
`php artisan widgets:types`	Regenerate widget TS types from the registry
`php artisan media:prune`	Clean orphaned media files/rows
`composer ide`	IDE helpers + all generated types
`php artisan import:wordpress`	WP XML import → posts + JSON pages

5. Security posture

- SVG uploads blocked (script injection); MIME whitelist enforced server-side
- Setting secrets encrypted at rest; strict `PUBLIC_SETTINGS` whitelist is the only path a setting reaches the browser
- Honeypot on public forms; auth rate limiting; activity log on auth events
- Roles/permissions enforced server-side on routes, nav, and search results

6. Hosting profile (shared-hosting friendly)

- MySQL only, no Redis required: cache, sessions, queue, and response cache all on the database
- Inode-frugal: response cache in DB rows not files, daily log rotation (14-day retention), scheduled expired-cache-row pruning
- GitHub Actions deploy via rsync; `.htaccess` handles compression + expiries

---

How a client site gets built (suggested workflow)

1. **Clone + init**: `php artisan template:init` → site name, admin user, seeded content
2. **Toggle modules**: enable only what the client needs (e.g. blog + testimonials + FAQs, no careers)
3. **Theme**: set brand color, font, radius in Settings → Theme
4. **Build the frontend**: override/extend the widget components in `resources/js/Components/Widgets/` with the client's design (see `agents/skills/` — especially `widgets`, `page-content`, `launch-readiness`)
5. **Content**: create pages in the widget editor, set menus + header/footer, write posts
6. **SEO**: fill the SEO settings tab (title template, LocalBusiness, analytics), per-page meta
7. **Launch gate**: `pnpm build` + `php artisan optimize` + `php artisan template:doctor` + Lighthouse pass per the `launch-readiness` skill; flip `SEO_INDEXABLE=true`

Universal frontend-build prompt

The exact prompt below (canonical copy: `docs/FRONTEND-PROMPT.md`) kicks off step 4 of the workflow above. Paste it into a fresh AI agent session running inside a clone of this repo (after `template:init`), replacing every `{{PLACEHOLDER}}` with the client's real values — delete any optional line that doesn't apply.

```
You are building the custom public frontend for a client website on top of the
webTemplate universal CMS backend (Laravel 13 + Vue 3 + Inertia v3 + Tailwind v4,
already set up in this repo). The backend is generic and stays untouched — your
job is ONLY the public-facing look: widget components, the public layout, theme,
and page content.

## Client brief

- Client / site name: {{CLIENT_NAME}}
- Industry / what they do: {{INDUSTRY_AND_SERVICES}}
- Target audience: {{TARGET_AUDIENCE}}
- Design direction: {{DESIGN_DIRECTION}}  (e.g. "minimal & premium, lots of whitespace", "bold &
colorful", "corporate & trustworthy")
- Reference sites the client likes: {{REFERENCE_URLS}}  (optional)
- Brand primary color (hex): {{BRAND_HEX}}
- Preferred font: {{FONT_NAME}}  (must exist on bunny.net fonts; otherwise pick the closest)
- Border radius feel: {{RADIUS}}  (sm = sharp / md = balanced / lg = soft)
- Logo / brand assets: {{ASSET_LOCATION_OR_UPLOAD_NOTE}}
- Pages the site needs: {{PAGE_LIST}}  (e.g. home, about, services, contact)
- Modules to enable: {{MODULE_LIST}}  (choose from: blog, testimonials, faqs, events, teams,
case_studies, careers, subscribers)
```

- Anything custom the design needs that the shipped widgets can't express: {{CUSTOM_SECTIONS_OR_NONE}}
- Copy/content source: {{CONTENT_SOURCE}} (e.g. "client doc at …", "write placeholder copy for the industry")
- Languages: single language, {{LANGUAGE}}

Before writing any code — read these, in order

1. `docs/FRONTEND.md` — the frontend contract (mental model, render path, rules). Follow it exactly.
2. `agents/skills/widgets/SKILL.md` — only if you add or change a widget TYPE.
3. `agents/skills/launch-readiness/SKILL.md` — before adding any public image or third-party script, and before calling the site done.
4. `resources/js/types/widgets.d.ts` — the exact data shape of every widget.

Hard rules (violating any of these breaks the backend contract)

- pnpm only, never npm. No new heavy dependencies without strong reason.
- Vue 3 `<script setup lang="ts">` + Tailwind v4 utilities. No Ziggy / route() helpers — links use literal root-relative paths ("/about", "/blog").
- Every public image renders through the `AppImage` component (srcset, width/height, lazy by default, `eager` only for the above-the-fold hero image).
- The Hero widget owns the page's single H1; every other widget starts at H2.
- Style with the theme tokens (`--color-brand-*` steps, `--font-sans`, radius token) — never hard-code the brand hex in components. The admin Theme settings must keep working: changing the color in the admin restyles the site.
- Do NOT touch: controllers, `WidgetDataResolver`, `config/widgets.php` field contracts of existing types, the `PUBLIC_SETTINGS` whitelist, permissions, migrations, or anything under `app/` — unless step 6 (new widget type) explicitly requires a registry entry + resolver case.
- SEO is handled globally (`seo` shared prop → PublicLayout `<Head>`): never emit your own `<title>`/meta/canonical in page or widget components.
- Header/footer chrome comes from shared Inertia props: `layout` (header/footer JSON), `menus` (nested tree), `settings` (whitelisted keys), `siteLogo`. Consume them — don't invent parallel config.

The work, in order

1. **Theme**: set `theme_primary_color` = {{BRAND_HEX}}, `theme_font` = {{FONT_NAME}}, `theme_radius` = {{RADIUS}} (Settings → Theme, or seed/update the `site_settings` rows). Verify both admin and public pick it up.
2. **Modules**: enable exactly {{MODULE_LIST}} from /admin/modules (or via the modules table); disable the rest so the sidebar and sitemap stay clean.

3. **PublicLayout** (`resources/js/Layouts/PublicLayout.vue`): restyle the header (logo via `siteLogo` + `AppImage`, nav from `menus.header`, CTA from `layout.header`), the footer (columns from `layout.footer`, socials from `settings`), and the mobile nav to match the design direction. Keep the existing `<Head>`/SEO and consent logic intact.
4. **Widget components** (`resources/js/Components/Widgets/*.vue`): restyle every shipped widget the site will use (hero, rich_text, feature_grid, stats, cta, image, gallery, contact_form, plus the collection widgets for the enabled modules: testimonials, faqs, team, latest_posts). Keep each component's `data` prop typed from `@/types/widgets` and keep rendering every registry field — editors rely on all of them working. Empty/missing data must render nothing, never a broken block.
5. **Blog & collection pages** (only for enabled modules): restyle `Pages/Public/Blog/{Index,Show}.vue`, category archive, and `Careers`/`CaseStudies` pages to match. Body HTML stays `v-html` output of the block editor — style it via typography classes.
6. **Custom sections** ({{CUSTOM_SECTIONS_OR_NONE}}): only if a design section truly can't be expressed with existing widgets, add a NEW widget type following `agents/skills/widgets`: registry entry in `config/widgets.php` (serializable, every field has a placeholder) → `php artisan widgets:types` → `Components/Widgets/YourType.vue` → map in `DynamicPage.vue` → `WidgetDataResolver` case only if it needs DB rows. Prefer composing existing widgets over inventing new ones.
7. **Pages**: create/update `data/pages/{slug}.json` for {{PAGE_LIST}} — real widget stacks per the design (home = hero + proof + services + CTA + …), with copy from {{CONTENT_SOURCE}}. Fill each page's `seo` block (meta title ≤60 chars, description ≤160). Set up header/footer menus and `data/header.json` / `data/footer.json` via the admin (Menus, Header & Footer screens) or directly in the JSON.
8. **Responsive + polish pass**: mobile-first check of every page; loading `eager` only on the LCP hero image; no horizontal scroll; visible focus states; color contrast meets WCAG AA against the brand palette.

Definition of done — run all of it, fix until green

1. `pnpm build` — vue-tsc must pass with zero errors.
2. `php artisan optimize` — must stay clean.
3. `php artisan template:doctor` — all green.
4. Browser pass with `composer dev`: every page in {{PAGE_LIST}} renders correctly at mobile/tablet/desktop widths; the contact form submits and the enquiry appears in /admin (test one bad phone number, one valid); changing the theme color in /admin restyles the public site; every enabled collection

```
   widget shows real rows; drafts and unknown slugs 404.
5. Lighthouse on the home page per `launch-readiness` — Performance/SEO/Best
   Practices/Accessibility all green (≥90), noting the documented third-party
   exceptions.
6. Report: list every file you changed, every widget type added (if any), and
   anything from the brief you could not satisfy and why. Do not mark the site
   launch-ready — leave `SEO_INDEXABLE=false`; flipping it is a human decision.
```

See `docs/FRONTEND-PROMPT.md` for the placeholder cheat-sheet (what to put in each `{{…}}` slot).

Deliberately out of scope

Ecommerce (removed), comments, revisions/trash, post scheduling UI, media `media_id` FKs (URL strings for now), role-editor UI (roles are seeded), multi-language/hreflang, public routes for testimonials/FAQs/events/team (widgets cover them).